Research Proposal

How accurately do LLM-generated preconditions and postconditions capture the behavioural requirements expressed in natural-language programming tasks, and how effectively do they detect incorrect generated implementations?

Author: Anisia Kochetkova

Advisor: Paolo Ciancarini

Date: 29.09.2026

Innopolis University

1 Introduction and Research Problem

Large language models (LLMs) are increasingly used to write code. A developer can describe what they want in plain English, and the model produces a working program. This is convenient, but it comes with a problem. The code may look correct on a few test examples while still containing hidden bugs that only appear on unusual inputs. Normal testing cannot catch all such bugs, because tests only check a limited number of cases. To catch more bugs, we need a stronger way of describing what a program is supposed to do.

One such way is a *specification*. A specification states the conditions that must hold before a function is called (preconditions) and after it returns (postconditions). Unlike a test, a specification describes the behaviour of a function for all possible inputs, not just for a few examples. If a program satisfies its specification, it is much more likely to be correct. The problem is that writing specifications by hand takes a lot of time and requires special skills.

Recent work has asked whether LLMs can write specifications for us. The idea is simple: if a model can read a task description and produce a correct specification, then that specification can be used to check whether a piece of code is correct. Some studies have shown promising results. For example, Clover, a system built for the Dafny language, accepted 87% of correct programs and rejected all of the incorrect ones it was tested on. Other studies have looked at generating specifications for verification tools or property-based tests from natural language.

However, there are still important gaps. First, most existing work focuses on formal verification languages such as Dafny, Lean, or Verus. These languages require a lot of extra annotations and are not used in everyday programming. Second, very little work has looked at *executable* specifications in ordinary Python—that is, assertions or property-based tests that can be run directly with standard tools, without a verifier. Third, no one has systematically measured the trade-off between how strict a specification is and how often it wrongly rejects correct code. A specification that is too weak will accept incorrect programs, while one that is too strict will reject correct ones.

This thesis addresses these gaps. I build a benchmark of small Python tasks with natural-language descriptions, correct implementations, and deliberately introduced faults. I then evaluate whether specifications generated by LLMs—preconditions, postconditions, and property-based tests—can reliably tell correct code apart from incorrect code.

2 Research Question

The central research question of this thesis is:

> **To what extent can executable specifications generated by LLMs from natural-language Python programming tasks reliably detect incorrect implementations while accepting correct ones?**

In other words, the goal is to determine whether specifications produced by large language models can serve as a dependable automated oracle for judging the correctness of code. The question has two dimensions: how well the specifications capture what a function is supposed

to do, and how well they distinguish correct implementations from incorrect ones. These two dimensions are addressed by two sub-questions.

RQ1. Accuracy of requirement capture

The first sub-question focuses on the relationship between the natural-language task description and the generated specification. When an LLM reads a task description, it must identify the behavioural requirements that any correct implementation must satisfy. These requirements include, for example, the allowed range of inputs, the expected type and meaning of the output, the behaviour on boundary cases, and whether the input may be modified.

RQ1 asks how faithfully the generated specification reflects these requirements. A specification is considered accurate if every requirement stated in the description is represented in the specification, and if the specification does not introduce additional constraints that were never implied by the description. Two metrics are used to answer RQ1:

- **Valid specifications %** measures how many generated specifications are syntactically and semantically valid, that is, they can be executed without manual repair.
- **Requirement coverage** measures how many of the stated requirements are actually reflected in the generated specification.

Together, these metrics describe how much of the intended behaviour the LLM was able to capture from the natural-language description alone.

RQ2. Effectiveness of fault detection

The second sub-question focuses on how well the generated specification distinguishes correct implementations from incorrect ones. For each task, the benchmark provides a set of implementations: some are correct, and some contain deliberate faults, either written by hand or produced automatically by mutation. A specification is useful if it accepts every correct implementation and rejects every faulty one.

RQ2 asks how often this happens in practice. Because specifications may be too weak or too strict, two types of errors are possible: a specification may accept a faulty implementation, or it may reject a correct one. Four metrics are used to answer RQ2:

- **False acceptance** measures how often a specification accepts an incorrect implementation.
- **False rejection** measures how often a specification rejects a correct implementation.
- **Mutation score** measures how many mutation-generated faults are detected by the specification.
- **Agreement with hidden tests** measures how closely the verdict of the specification matches the verdict of an independent set of reference tests.

Together, these metrics describe how reliable the generated specification is as a tool for detecting incorrect code.

Control metrics

In addition to the metrics above, two control metrics are recorded. They do not answer the research questions directly, but they are necessary to assess whether the approach is practical:

- **Cost** measures the number of tokens and the monetary cost required to generate a specification.
- **Execution time** measures how long it takes to run a specification against an implementation.

Mapping of metrics to research questions

Metric	Research question
Valid specifications %	RQ1
Requirement coverage	RQ1
False rejection	RQ2
False acceptance	RQ2
Mutation score	RQ2
Agreement with hidden tests	RQ2
Cost	Control
Execution time	Control

3 Novelty and Contributions

To the best of our knowledge, no existing work combines executable Python specifications, a mutation-based fault model, and a game-themed task set in a controlled benchmark. This work fills that gap.

The novelty of this work lies in combining three elements:

1. *Executable* Python specifications (assertions and Hypothesis properties) rather than formal-verification annotations.
2. A *mutation-based* fault model that provides ground-truth labels for specification effectiveness.
3. A *game-themed* task set that yields natural invariants and edge cases suitable for property-based testing.

Planned contributions:

1. A **benchmark of n game-themed Python tasks** with natural-language descriptions, one or more correct implementations, hand-written faults, and mutation-generated faults.
2. **Two LLM-based specification strategies**: (i) preconditions, postconditions, and invariants as Python assertions; (ii) property-based tests using Hypothesis.
3. A **comprehensive evaluation** using mutation score, false acceptance, false rejection, requirement coverage, and agreement with hidden reference tests.

4. An **open-source harness** for reproducible evaluation of LLM-generated specifications against faulty code.

It is important to note that passing executable checks provides evidence of correctness, not a formal proof.

4 Related Work

Several lines of research are closely related to this thesis, but none of them combine the specific elements proposed here.

LLM specification generation. Recent work has explored whether large language models can write specifications. NL2Contract demonstrated that LLMs can translate natural-language docstrings into formal preconditions and postconditions. Clover applied similar techniques to the Dafny verification language, checking consistency between code, annotations, and docstrings, and achieved an 87% acceptance rate for correct programs. Verus-SpecGym evaluated specification autoformalization in Rust and found that even frontier models often fail to write faithful specifications for problems they can solve in code.

Comparison with NL2Contract and Agentic Property-Based Testing. While NL2Contract and Agentic Property-Based Testing (PBT) are foundational, our approach differs fundamentally across four key dimensions:

- **Inputs:** NL2Contract uses formal code and docstrings as input. Our work uses *informal natural-language task descriptions* (game rules), simulating a more realistic scenario where developers describe what they want in plain English.
- **Specification format:** NL2Contract generates formal verification contracts, and Agentic PBT generates executable test suites. In contrast, our work asks the LLM to generate *executable Python assertions and Hypothesis properties* that act as preconditions and postconditions. This format does not require a heavy formal verifier.
- **Benchmark:** Existing benchmarks focus on code generation or verification success. Our benchmark introduces *40 game-themed tasks* paired with systematically generated faults (mutants) to serve as a controlled ground truth.
- **Evaluation:** Prior work evaluates verification success or test coverage. Our evaluation measures the *fault detection capability* of the generated specifications, specifically looking at False Acceptance and Mutation Score to see if the specifications can catch the bugs.

Executable specifications and testing. Property-based test generation has been explored in other domains. Mutation testing has been widely used to assess the quality of LLM-generated test suites. Prior work in this area shows that model selection matters more than prompt design, and that executability is often the binding constraint. We build on this by using mutation score as a primary metric for evaluating specification effectiveness.

Benchmarks for code generation. Standard benchmarks like MBPP and HumanEval provide natural-language descriptions with simple test cases, but they lack formal specifications and systematic fault injection.

In summary, prior work has established two things: LLMs can generate specifications in formal languages such as Dafny and Verus, and mutation testing is useful for evaluating test suites. However, no existing work systematically evaluates executable Python specifications generated by LLMs against mutation-generated faults in a controlled, game-themed benchmark. This thesis fills that gap.

5 Goal of the Thesis

The goal of this thesis is to find out whether specifications written by large language models can be used to check if a piece of Python code is correct. In other words, can an LLM read a task description, write a specification, and then use that specification to tell correct code apart from incorrect code?

To answer this, I will build a benchmark of small Python tasks and measure how well the generated specifications work. I will look at four things: whether the specifications are valid, how many faults they can detect, how often they wrongly reject correct code, and how much they cost to generate.

The result of this work will be three things. First, a reusable benchmark and evaluation harness that other researchers can use. Second, a comparison of two different ways to ask an LLM for a specification. Third, practical advice on when specifications can replace or support normal testing in projects where code is written with the help of LLMs.

6 Detailed Work Plan and Schedule

The thesis is planned over a period of approximately nine months, from early September to May. The work is divided into six phases, each with a clear deliverable and a deadline. Two chapters of the thesis must be written by 1 November, two more by March, and the remaining chapters by May, leaving a buffer for final revisions before the defense.

Phase 1: Benchmark design (early September – October). Select 40 game-themed Python tasks across 17 domains: tic-tac-toe, chess, checkers, sudoku, minesweeper, fifteen puzzle, 2048, snake, tetris, Conway's Game of Life, map, battleship, reversi, dice, HP/damage, experience, inventory. For each task, write a natural-language description (5–8 lines), define the function signature, and specify 3–4 behavioural requirements. Produce the JSON schema and one complete example task.

Phase 2: Implementations and faults (October – December). In parallel with writing the first two thesis chapters, write one or two correct implementations per task in distinct styles. Generate faulty implementations via hand-written typical faults and mutation operators using Cosmic Ray. Write hidden reference tests and validate that they pass on correct implementations and fail on all faulty variants. Exclude equivalent mutants.

Phase 3: Prompts and specification generation (January). Implement two LLM prompting strategies: (A) direct generation of Python `assert` statements; (B) generation of Hypothesis `@given` property-based tests. Use a fixed set of frontier models with temperature 0 and 3 samples per task. Run a pilot on five tasks to validate the pipeline before scaling to all 40 tasks.

Phase 4: Evaluation harness and metrics (February). Build a harness that loads each task, executes generated specifications against all implementations, and records pass/reject decisions. Compute specification validity rate, false rejection rate, false acceptance rate, mutation score, requirement coverage, agreement with hidden tests, token cost, and execution time. Run the full experiment on all 40 tasks.

Phase 5: Analysis and writing (March – April). Qualitative analysis of failure modes: which requirements are most often omitted, which fault types escape detection, and whether false rejections arise from over-specification or ambiguity in natural language. Write the remaining thesis chapters and prepare the open-source artifact (benchmark, prompts, harness, logs).

Phase 6: Final revisions (May). Polish the thesis, prepare the defense presentation, and submit the final version. This phase is intentionally left with some slack to accommodate unexpected delays.

Period	Phase	Deliverable
Early Sept – Oct	Benchmark design	40 tasks, NL descriptions, JSON schema
Oct – Dec	Implementations and faults	correct + faulty + mutants + hidden tests
January	Prompts and generation	2 prompts, pilot + full specification set
February	Harness and metrics	harness, full run, metric tables
March – April	Analysis and writing	figures, error analysis, thesis chapters
May	Final revisions	polished thesis, defense presentation

Chapter deadlines

Deadline	Chapters	Content
1 November	Chapters 1–2	Introduction, Related Work
March	Chapters 3–5	Benchmark design, Methodology, Results
May	Chapters 6–7	Discussion, Conclusion

7 References

1. Verus-SpecGym: An agentic environment for evaluating specification autoformalization. arXiv:2605.26457.
2. Clover: Closed-loop verifiable code generation. SAIV 2024. PDF.
3. TESTDAFNY110: A curated dataset for automatic generation of formal specification and verification annotations using LLMs and test oracles. arXiv:2601.12845.
4. Agentic property-based testing: Finding bugs across the Python ecosystem. arXiv:2510.09907.
5. NL2Contract: Beyond postconditions – Can large language models infer formal contracts for automatic software verification? arXiv:2510.12702.